FIB

AHLT Lab Project - Final Report

Task 2: Drug–Drug Interaction (DDI) Extraction

Feature-based ML vs. Neural Networks vs. Large Language Models on SemEval-2013 Task 9

Table of Contents

Lab Group 10 - authors and primary responsibility:

Member	Primary focus across the three systems
Ralph Khairallah (Exchange)	Feature engineering, fine-tuning campaigns, error analysis
Francesco Dorati (Exchange)	Neural architecture sweeps, evaluation, figures

Repository: `github.com/francesco-dorati/AHLT-project`
Shared task: `SemEval-2013 Task 9 (DDIExtraction)`
Date: June 13, 2026

---

1 Executive Summary

At a glance

This report covers **Task 2 of the AHLT lab project: Drug–Drug Interaction (DDI) extraction**, framed as a sentence-level classification problem on the SemEval-2013 Task 9 corpus. We build, tune and compare **three systems** on the *same* data with the *same* evaluator: a feature-based **Machine-Learning** classifier (2.1), a **Neural-Network** BiLSTM+CNN (2.2), and a **Large Language Model** fine-tuned with LoRA (2.3). The headline result is the *mirror image* of Task 1 (NERC): on this relational task the classical ML system **wins** (test macro-F1 = 66.8 %), the neural network is within 1.2 pp (65.6 %), and the fine-tuned 3B LLM trails both by ≈27 pp (39.8 %). The dominant difficulty throughout is the **≈85 % `null` class imbalance** and the fact that the discriminative signal lives in *syntax* (the dependency path between the two target drugs), information the ML model encodes directly through hand-crafted features but the LLM has to reconstruct from raw text.

Headline fact	Value
Task framing	sentence-level 5-class problem (advise / effect / int / mechanism / `null`)
Class imbalance	≈85 % of all pairs are `null` (no interaction)
Best system (test macro-F1)	**2.1 ML** two-stage on `mod_best2` = **66.8 %**
Neural network (test)	2.2 BiLSTM + relative-position embedding = 65.6 %
Fine-tuned LLM (test)	2.3 Llama-3.2-3B, LoRA r=32, `prompts02` = 39.8 %
Rule-based floor (test)	2.0 cue-word baseline = 26.9 %
Biggest lever, ML	a two-feature combo + a two-stage classifier (the largest single jump, +1.1 pp test)
Biggest lever, NN	input representation (prefix + entity-type marker, +10 pp)
Biggest lever, LLM	LoRA rank *and* prompt quality (∼+5 pp each)
Headline cross-task lesson	on a *relational* task, classical NLP clearly beats a small LLM

Table 1: Headline numbers for the three delivered DDI systems. All test-set figures use the shared `util/evaluator.py DDI` script and report macro-F1 (M.avg) over the four positive classes.

2 Introduction

2.1 Context

Drug–drug interactions are a leading cause of avoidable adverse clinical events, and most of the evidence for them is locked in unstructured biomedical text: drug labels, case reports, and scientific abstracts. Task 1 of this project located and typed the drug *mentions* (NERC). Task 2 takes the next step: given a sentence and two drug entities already marked in it, decide whether the sentence *states* that the two drugs interact, and if so which *type* of interaction it is. Crucially, as the lab brief stresses, we are not deciding whether two drugs interact in reality, but whether the *target sentence says so*. DDI is therefore a **sentence-level relation-classification** problem, not the sequence-labelling problem Task 1 was.

This change of framing is what makes Task 2 the more interesting half of the project, and it is worth stating up front why it is harder. First, the label distribution is brutally skewed: roughly **85 % of all candidate pairs are `null`** (the sentence asserts no interaction for that pair), so a model that simply answers "no interaction" everywhere already scores 85 % accuracy while being useless, which is exactly why the evaluator ignores `null` and we report macro-F1 over the four positive classes. Second, the discriminative signal is *relational and syntactic*: it lives in the dependency path connecting the two target drugs and in the trigger verbs along that path ("*X potentiates Y*", "avoid co-administration of *X with Y*"), not in the surface form of any single token. A model that cannot

represent "which words sit *between* and *syntactically connect* the two drugs" is working blind.

2.2 Objectives and how this report reads

We deliver and compare three DDI systems spanning the main paradigms in modern NLP, exactly as the lab specification requires:

- **System 2.1, feature-based Machine Learning**: a Maximum-Entropy (logistic-regression) classifier over hand-crafted lexical and *syntactic* features (dependency paths, clue verbs, entity-type pairs), extended with a two-stage null-vs-positive router.
- **System 2.2, Neural Networks**: the shipped BiLSTM+CNN sentence classifier, improved through input-representation engineering (prefix / suffix / entity-type / relative-position channels), architecture variants, hyperparameter sweeps and a multi-seed robustness audit.
- **System 2.3, Large Language Models**: a generative single-label classifier, evaluated both few-shot (no training) and after LoRA fine-tuning of 4-bit-quantised Llama-3.2-3B and Qwen-2.5-3B, with prompt-design, few-shot and fine-tuning-recipe ablations.

For every system we follow the same narrative the deliverable asks for: *what we tried, why we tried it, what the variation did, and what we concluded*. We deliberately report negative results in full, they are as informative as the wins, and several of the most instructive findings (the seed lottery in 2.2, the prompt brittleness in 2.3) are negative or counter-intuitive. Throughout, we draw explicit comparisons to Task 1 (NERC); the closing section (7) gives the required cross-system comparison along the brief's four axes , *development effort*, *computational cost*, *explainability and controllability*, and *resulting performance*, and explains the central surprise of the whole project: **the same LLM recipe that won Task 1 loses Task 2**.

3 Data, Evaluation, and Reproducibility

3.1 Dataset

The data is the DDI Extraction 2013 corpus (SemEval-2013 Task 9), combining **DrugBank** drug-label text and **MedLine** research abstracts. Sentences come with gold drug entities; every *ordered pair* of entities within a sentence is a classification instance whose gold label is one of four interaction types or `null`:

- **advise**: a recommendation or warning about combining the drugs ("*should be avoided*").
- **effect**: a clinical effect / pharmacodynamic outcome of the combination.
- **mechanism**: a pharmacokinetic mechanism (absorption, metabolism, clearance).
- **int**: a generic, untyped interaction mention ("*X interacts with Y*").
- **null**: the sentence does *not* assert an interaction for this pair.

Split	Sent.	Pairs	advise	effect	int	mech.	null
Train	5 343	22 793	695	1 439	201	1 016	19 442
Devel	1 477	4 877	143	316	43	261	4 114
Test	1 455	5 838	209	293	40	344	4 952
% null			train 85.3 %		devel 84.4 %		test 84.8 %

Table 2: DDI-2013 corpus statistics. The `int` class is extremely rare (40–201 pairs) and `null` dominates every split, which is the central modelling challenge of the task. Contrast Task 1, where the rare class (*drug_n*) was ∼4 % of entities; here `int` is <1 % of pairs.

3.2 Evaluation protocol

The provided `util/evaluator.py DDI` scores predictions at the *pair* level. A prediction is correct only if both the "interacts" decision and the interaction *type* match the gold label. The evaluator

reports precision, recall and F1 *per positive class*, plus two aggregates: **macro-F1 (M.avg)**, averaging the four positive-class F1s equally, and **micro-F1 (m.avg)**, pooling all positive predictions. `null` pairs are excluded from F1 by construction (predicting `null` can only cost recall on the positives, never earn credit). Because the four positive classes differ wildly in frequency, **macro-F1 is the primary metric** throughout this report, it is the number a report must defend, since micro-F1 is dominated by the two common classes (`effect`, `mechanism`). This is the identical protocol to Task 1, swapping entity spans for ordered pairs, so the cross-task comparison in Section 7 is on equal footing.

3.3 Reproducibility

Each system lives in its own directory (`code/2.1.DDI-ML/`, `2.2.DDI-NN/`, `2.3.DDI-LLM/`) with a full experiment log in `NOTES.md`; every code change is tagged `[MOD-2.x]` in source with a one-line reason. System 2.1 runs entirely on CPU; 2.2 and 2.3 run on the UPC `Boada` GPU cluster via `sbatch`. Every figure in this report is regenerated from the logged `.stats` files by the `report/generate_plots_2_*.py` scripts, so each plotted number is traceable to a committed result file.

4 System 2.1 - DDI with Machine Learning

System 2.1 is the feature-based pipeline. We follow the brief's suggested order, establish a reference, then engineer features (with "special attention to syntax-based features"), then sweep algorithms and hyperparameters, and we mirror the methodology we used for the Task 1 CRF so the two ML systems are directly comparable.

4.1 Baseline (2.0) and why it is the floor

The rule-based starting point classifies a pair by matching ∼50 hand-curated cue words found between the two drugs. Its per-class scores are revealing:

Split	advise	effect	int	mech.	M-F1	m-F1
Devel	0.0	12.3	70.7	5.8	22.2	13.1
Test	20.1	25.2	50.0	12.3	26.9	20.8

Table 3: Rule-based 2.0 baseline. It *nails* `int` (its cue list contains "interact") but collapses on `advise` (0 % on devel), where its cues are over-specific drug names. Test M-F1 = 26.9 is our floor.

The lesson the baseline teaches is that **a fixed cue list cannot generalise**: it works only where the trigger vocabulary happens to be in the list. Everything in System 2.1 is about replacing that brittle list with *learned* weights over a much richer, partly *syntactic* feature space.

4.2 Reference classifier: MEM vs. SVM

The provided ML reference replaces cue-matching with a classifier over a rich shipped feature set, entity types, the `same-drug` flag, the lowest-common-subsumer (LCS) lemma/PoS, **nine dependency-path variants**, words/lemmas in the path, and four pattern families (verb-LCS, verb-function, words-in-between, words-outside). The brief asks us to compare learning algorithms, so we first decided which classifier to carry forward.

Classifier (devel)	advise	effect	int	mech.	M-F1	m-F1
MEM (LogisticRegression)	62.6	65.4	69.2	60.0	**64.3**	63.2
SVM (rbf, default)	49.2	60.2	61.5	49.0	55.0	54.3
SVM (linear, C=0.5)	-	-	-	-	60.5	60.0

Table 4: Reference classifiers on devel. MEM beats SVM on *every* class; the rbf-SVM is heavily precision-biased (recall <45 % everywhere). A linear kernel with lower C recovers to 60.5 but still trails MEM by 3.8 pp.

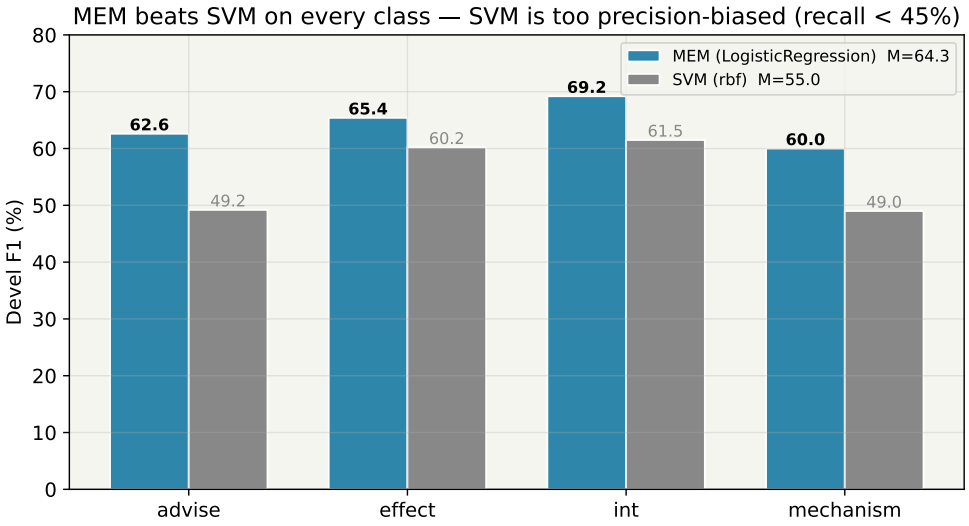


Figure 1: MEM vs. SVM per-class F1 on devel. The shipped feature space is high-dimensional and sparse, exactly logistic regression's regime, whereas the rbf-SVM under-fits the positive classes.

Findings.

- **MEM wins because the feature space suits it.** With thousands of sparse binary features (dependency-path strings, pattern lemmas), a linear maximum-entropy model fits cleanly while the rbf kernel's similarity assumption is wrong for one-hot path features.
- **SVM is precision-biased.** The rbf-SVM's recall is below 45 % on every class, it abstains on anything ambiguous. A linear kernel with a smaller C closes part of the gap but never catches MEM.
- **This mirrors Task 1's algorithm choice.** There the CRF won because it models BIO *label sequences*; here MEM wins because the signal is already in the (syntactic) features and only a good linear separator is needed. **Decision: MEM is the carrier for all feature engineering.**

We also confirmed default regularisation is right: a C sweep gave C=1 best (61.1 at C=0.1, 64.3 at C=1, 63.2 at C=5), and `class_weight=balanced` *collapsed* to M=53.2, balancing across the 5-way training set over-corrects because the evaluator discards `null` anyway.

4.3 Feature-engineering experiments

With MEM fixed, we added candidate feature mods one at a time on top of the reference, re-extracting features and re-training after each, exactly as we did for the Task 1 CRF. The brief asks for *special attention to syntax-based features*, so our candidates are mostly syntactic or trigger-oriented.

Mod	What it adds	M	ΔM
ref	shipped feature set	64.3	-
mod1	distance + sentence-position / length buckets	62.4	−1.9
mod2	pharmacology clue-verbs (lemma + position vs. the pair)	64.2	−0.1
mod3	third-entity context (other entities in sentence / in path)	64.0	−0.3
mod4	combined entity-type pair feature (`typeE1_typeE2`)	64.1	−0.2
mod5	negation cues near the LCS / in the path	63.0	−1.3
mod_best	**mod2 + mod3 + mod4**	**65.2**	**+0.9**
mod_best2	**mod2 + mod3** (drop mod4)	**65.4**	**+1.1**
mod6	lemma bigrams in the words-in-between region	64.6	−0.8†
mod7	LCS subtree shape (LCS lemma + child deps)	65.1	−0.3†

Table 5: Additive feature mods on devel macro-F1. †mod6/mod7 are measured on top of `mod_best2`; both hurt it. Every *single* mod is neutral-to-negative, but the `mod2+mod3` combination synergises.

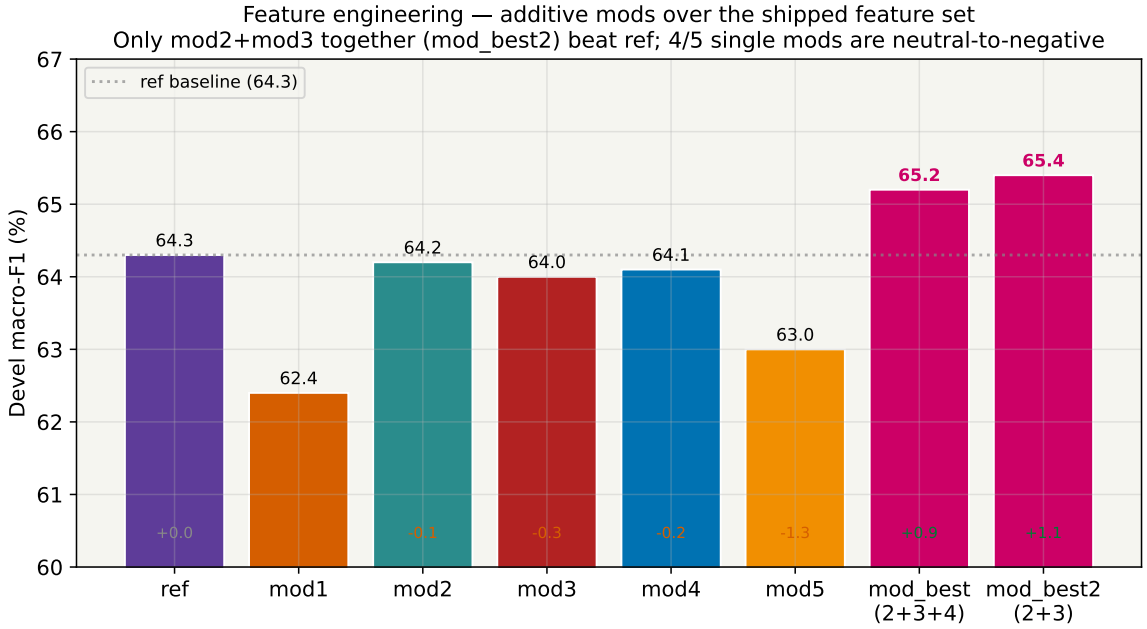


Figure 2: Feature-mod sweep on devel macro-F1. The shipped feature set is already well-tuned: isolated additions dilute a sparse linear model, but a carefully chosen *combination* (`mod_best2`) wins.

Findings.

- **Distance / length buckets hurt (−1.9).** mod1's coarse position features add noise the dependency-path features already encode more precisely, the path length *is* the distance, with syntax attached.
- **Clue-verbs and third-entity context are each ≈neutral alone but synergise.** mod2 (−0.1) and mod3 (−0.3) look like dead weight individually, yet `mod2+mod3` gains +1.1 pp. The trigger-verb signal becomes useful precisely *when* the model also knows how many other drugs sit in the path (third-entity context disambiguates "which pair does this verb describe?").
- **Negation cues hurt (−1.3).** A bag of "not / no / without" lemmas without scoping confuses more than it helps; proper negation handling would need to know *what* is negated.
- **mod4 helps the triple but hurts the pair.** An ablation (next paragraph) shows that dropping mod4 from `mod_best` actually *raises* devel M to 65.4, so the devel-selected champion is `mod_best2`

(mod2+mod3 only).

Key observation - combinations beat individuals

The single most important finding of the feature campaign is that **a combination of two individually-useless features wins**. This is the same lesson Task 1 taught with biomedical patterns and char n-grams, and it is the reason we do additive sweeps *and* ablations rather than trusting single-feature deltas. A leave-one-out ablation on `mod_best` confirms it: dropping mod2 costs −0.7, dropping mod3 costs −0.8, but dropping *mod4 gains* +0.2, so the real champion is `mod_best2`.

4.4 Algorithm and hyperparameter sweep

The brief asks for a hyperparameter exploration. With `mod_best2` features fixed, we swept solver, penalty and C. We expected little , and that is exactly what we found.

Solver	Penalty / C	M	m	Note
lbfgs	l2, C=1	65.2	64.6	default
lbfgs	l2, C=1.5	64.4	64.1	over-fit
saga	l2, C=1	**65.5**	64.7	best single-stage
saga	l2, C=5	64.1	63.4	under-regularised
saga	l1, C=1	63.8	61.8	sparsity hurts
saga	elasticnet, C=1, ρ=0.3	64.6	63.7	-

Table 6: Hyperparameter sweep on `mod_best2`. The best variant (saga l2 C=1, M=65.5) is +0.1 pp over the lbfgs default. l1 sparsity actively hurts.

Finding: hyperparameter tuning ≈ 0. The best variant beats the default by a single decimal. *This is the same conclusion Task 1 reached*, on sparse hand-crafted features, the representation is the lever, not the optimiser. l1 regularisation hurts because it zeroes out informative-but-rare path features.

4.5 The two-stage classifier - the largest single jump

The flat 5-way MEM fights two problems at once: the null-vs-positive imbalance *and* the inter-positive boundary. We hypothesised that *separating* these would help, and decomposed the classifier: **stage 1** is a binary null-vs-positive router (logistic regression); **stage 2** is a 4-way classifier trained on positive-only data. A pair is sent to stage 2 only when stage 1's P(positive) exceeds a threshold tuned on devel.

Threshold t	devel M	devel m	test M	test m
0.20	63.3	62.1	-	-
0.30	64.8	63.9	-	-
0.35	65.3	64.7	-	-
0.37	**66.0**	**65.2**	**66.6**	**62.4**
0.40	65.8	65.2	66.6	62.5
0.45	64.7	64.7	-	-
0.50	64.4	64.8	-	-

Table 7: Stage-1 threshold sweep (no class-balancing). The optimum is well below 0.5.

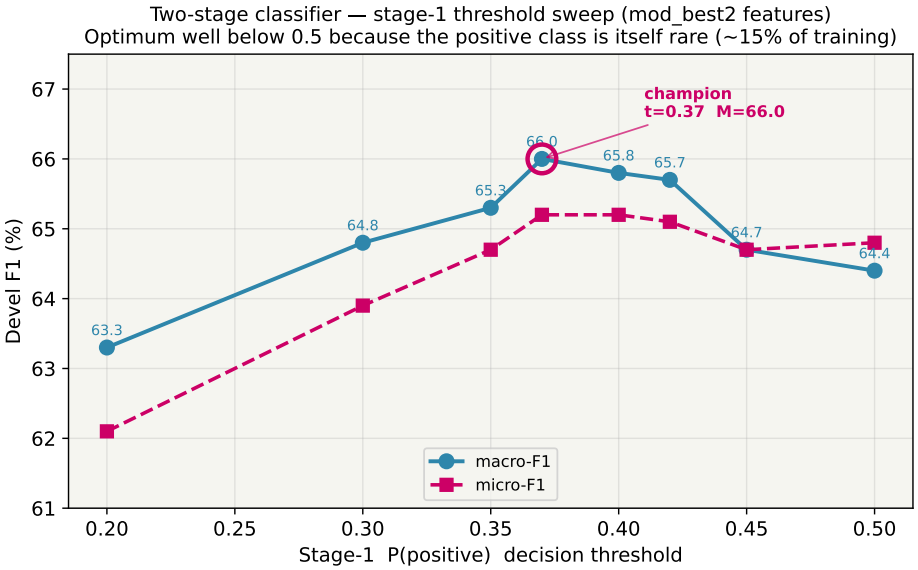
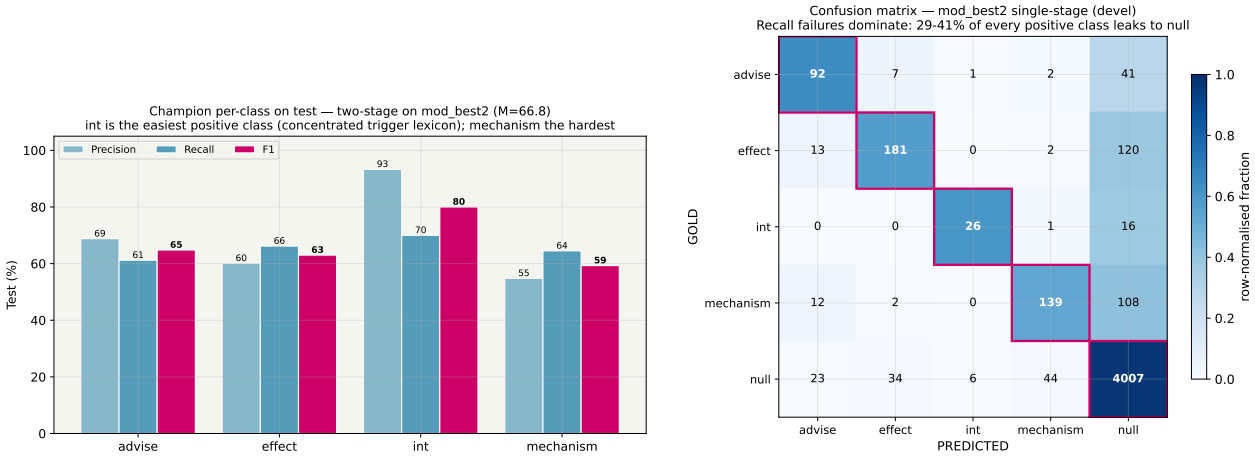


Figure 3: Stage-1 threshold sweep on devel. The optimum (t = **0.37**) sits below 0.5 because the "positive" class is itself rare (≈15 % of training pairs): demanding 0.5 probability would reject too many borderline true positives.

Findings.

- **The two-stage router is the System 2.1 champion**: devel M = 65.9, **test M = 66.8**, a +1.1 pp test gain over single-stage `mod_best2` (65.7) and the largest single jump in the whole campaign. (The sweep table reports the development-time figures, devel M = 66.0 / test M = 66.6 at t=0.37; the canonical re-train used for the cross-system comparison gives 65.9 / 66.8, the ±0.2 being re-run variance.)
- **The optimal threshold is 0.37, not 0.5.** Because stage 1's "positive" class is rare, a 0.5 cut-off is too conservative; lowering it to 0.37 trades a little precision for a lot of recall on every class.
- **Class-balancing stage 1 over-corrects.** With `class_weight=balanced` the best devel M is only 63.4 (vs 66.0), the same over-correction we saw on the flat MEM.

4.6 Best results, per-class and error analysis



(a) Champion per-class P/R/F1 on test. (b) Confusion matrix (`mod_best2`, devel).

Figure 4: System 2.1 champion analysis. `int` is the easiest positive class (test F1 = 80.0, a concentrated trigger lexicon); `mechanism` is the hardest (diffuse phrasing). The confusion matrix shows the dominant error is *recall* failure, 29–41 % of every positive class leaks to `null`, not inter-positive confusion, which is small.

Reading the confusion matrix. The off-diagonal mass is overwhelmingly in the last *column* (positives misrouted to `null`: 41 advise, 120 effect, 16 int, 108 mechanism), not in the inter-positive cells (at most 13 effect→advise). **The classifier separates the four positive classes well; it simply does not *activate* them often enough**, which is precisely the imbalance problem the two-stage router was built to attack. The largest false-positive direction is `null`→`mechanism` (44), because mechanism vocabulary ("induce", "inhibit") overlaps neutral pharmacology prose.

What the learned weights say (explainability). Inspecting the single-stage MEM weights confirms the model is linguistically sensible: the strongest single feature in the entire model is `same-drug` (+2.76, a drug mentioned twice is almost never a real interaction), `advise` is driven by modal constructions (`lcsCH=should`, `pat_clue=recommend`), and `mechanism` by a transitive dependency shape plus pharmacokinetic vocabulary (`pat_wout=metabolism`, `absorption`). One red flag: the `int` weights include a memorised training token ("MIVACRON"), a small over-fit on a 201-example class.

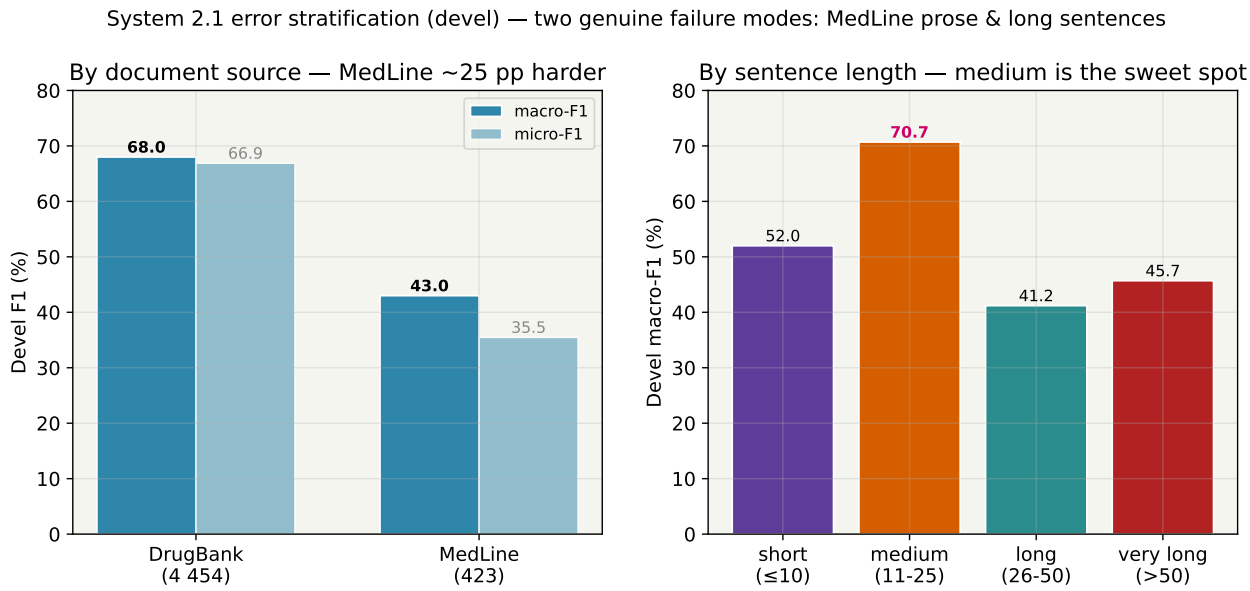


Figure 5: Error stratification on devel. **DrugBank pairs are ≈25 pp easier than MedLine** (standardised drug-label phrasing fits the clue-verb features; research-abstract prose does not), and **medium-length sentences are the sweet spot**, both very short (too few words for verb features to fire) and long (>25 words, noisy dependency paths) sentences are harder.

4.7 Deep dive: five extensions, none of which survive test

After the headline we pushed five further refinements, partly to be sure we had not stopped early. Every one tells the same story.

Extension	devel M	test M	Verdict
two-stage (champion)	65.9	**66.8**	keep
+ stage-2 class-balance	66.2	66.0	devel up, test down
+ train+devel re-train	-	66.6	threshold mis-calibrates
+ per-class thresholds	**68.4**	65.3	overfits devel (−1.5 test)
+ mechanism trigger lexicon (mod8)	65.4	65.7	hurts test
+ single/two-stage ensemble (AND)	65.8	65.5	devel-best ≠ test-best

Table 8: Post-headline extensions. The per-class-threshold row is the cautionary tale: a spectacular +2.5 pp on devel evaporates to −1.5 pp on test.

Finding: at this data scale, devel improvements that add free parameters do not generalise. The four per-class thresholds have enough freedom to overfit the ∼700 devel positives; the single global threshold acts as an implicit regulariser. Re-training on train+devel *lowered* test M because the stage-1 probability calibration shifted and the devel-tuned threshold no longer matched, and we cannot honestly re-tune it without a second held-out set. **The plain two-stage classifier is the legitimate, devel-selected champion.**

4.8 Discussion and comparison to Task 1

2.1 vs. Task 1's CRF. Both ML systems reach their ceiling through *feature combinations*, not hyperparameters, and both find the shipped feature set already well-tuned (4/5 single mods neutral-to-negative here; similar in Task 1). The difference is structural: Task 1 NER needed a *sequence* model (CRF) for BIO transitions, whereas DDI needs a good *relational* feature set (dependency paths) and only a linear separator on top, so MEM, which lost to CRF on NER, *wins* here. The two genuine failure modes (MedLine prose, long sentences) are out-of-domain for the syntactic feature set and recur in all three DDI systems.

System 2.1 final: two-stage on `mod_best2`, lbfgs, t=0.37 → **test M-F1 = 66.8** (+39.9 pp over the rule-based 2.0). This is the strongest system in the entire project for DDI.

5 System 2.2 - DDI with Neural Networks

System 2.2 replaces hand-crafted features with learned representations. The question we are really testing is whether a neural network can *recover*, from word/lemma/PoS sequences alone, the syntactic signal that System 2.1 hand-codes through dependency paths.

5.1 Starting point: the shipped architecture

Despite being called "CNN" in the brief, the shipped network is a **BiLSTM-then-CNN hybrid**: three embedded input channels (lower-cased word, lemma, PoS) are concatenated, run through a BiLSTM(250→200), then a 1-D convolution and a global max-pool, and a linear layer projects to the 5 classes. Critically, the network sees *only* word/lemma/PoS sequences, it has **none** of the dependency-path features that carry most of the DDI signal in 2.1. The reference reaches devel M-F1 = 55.8 (val-acc 88.35 %), **≈8.5 pp below the ML reference** (64.3) and 10 pp below the eventual ML champion. That gap is the opportunity, and it already tells us where to look: *representation*, not depth.

ref (devel)	advise	effect	int	mech.	M-F1	m-F1
F1	69.0	63.0	40.5	50.5	55.8	59.4

Table 9: Shipped-network per-class on devel. It trails the 2.1-ML reference by −8.5 pp on M-F1 (55.8 vs 64.3), with the gap concentrated in `int` (40.5 vs 69.2) and `mechanism` (50.5 vs 60.0), the two classes that most need syntactic context.

5.2 Phase I - input-representation experiments

We first asked which extra input channels recover the missing signal. This turned out to be by far the largest lever in System 2.2.

Run	Inputs added	M	m	val-acc
ref	lc-form + lemma + pos	55.8	59.4	88.35
mod1	+ suffix-5 + entity-type	64.8	68.3	91.41
mod2	**+ prefix-3 + entity-type**	**65.8**	66.9	91.43
mod3	+ entity-type only	62.8	67.2	90.44
mod4	+ case-sensitive form	61.8	65.1	91.00
mod5	+ suffix + prefix + entity-type (all)	62.0	65.8	91.00
mod6	+ suffix-5 only (no etype)	61.0	64.1	90.59
mod7	+ prefix-3 only (no etype)	64.7	67.1	90.44

Table 10: Phase I input-representation sweep on devel. Prefix-3 plus an entity-type marker (`mod2`) is the winner. `mod8` (all four channels) was OOM-killed during training.

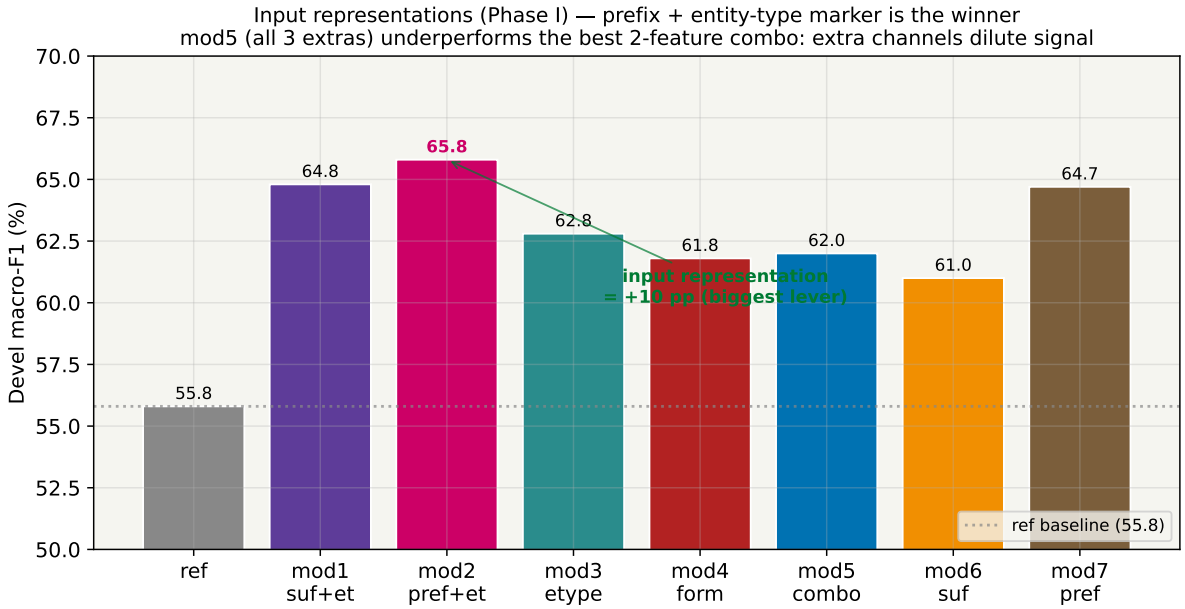


Figure 6: Phase I input-representation sweep. Adding a **prefix-3 channel plus an entity-type marker** (`mod2`) lifts macro-F1 by **+10 pp** over the shipped baseline, the single biggest improvement anywhere in System 2.2.

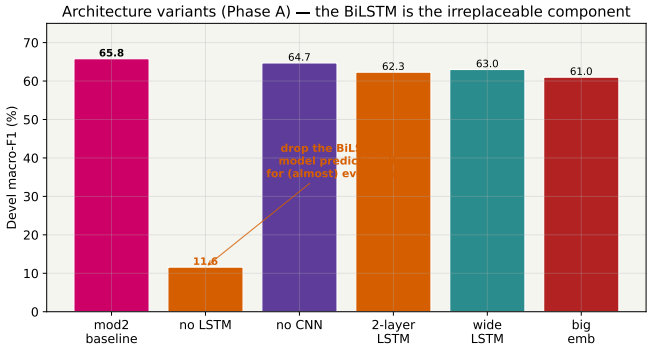
Findings.

- **Input representation is worth +10 pp.** The shipped network was starved of features, not architecture, adding prefix + entity-type closes most of the gap to 2.1-ML in one move. This is the *exact* lesson of Task 1's System 1.2, where PoS and prefix each gave ∼+9 pp.
- **Prefix carries most of the signal; entity-type adds a constant.** mod7 (prefix only) reaches 64.7; the entity-type marker on top (mod2) adds +1.1. The marker tells the network which tokens are `[DRUG1]`/`[DRUG2]`, a cheap, explicit hint.
- **Combining all extras *hurts* (mod5 = 62.0).** The best two-channel combo beats the all-channels combo: at this data scale, extra input channels dilute rather than enrich. Same overfitting-from-too-many-features pattern as 2.1's feature mods.

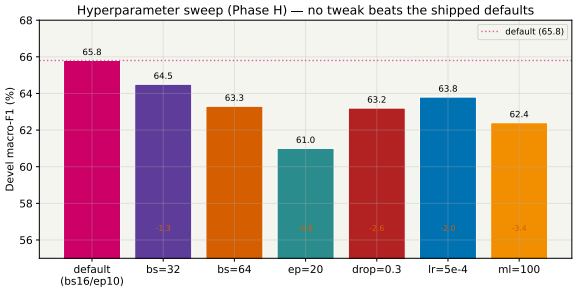
Confound, resolved. `mod1` and `mod2` were first run before an explicit `use_etype` flag existed, when the entity-type indicator was *always* on. The clean re-runs (`mod6` suffix-only, `mod7` prefix-only) isolate the channels and confirm prefix is the workhorse. We report the clean numbers and flag this so the +10 pp is not mis-attributed.

5.3 Phase A - architecture, and Phase H - hyperparameters

With `mod2` inputs fixed, we swept the architecture and then the hyperparameters, the two axes the brief names for the NN system.



(a) Architecture variants (Phase A).



(b) Hyperparameter sweep (Phase H).

Figure 7: Phase A/H on devel macro-F1. **Removing the BiLSTM collapses the model to 11.6** (it predicts `null` for almost everything); the CNN contributes only ≈1 pp. No depth/width change and no hyperparameter tweak beats the shipped defaults.

Architecture	M	Hyperparameter	M
mod2 baseline	65.8	default (bs16/ep10)	65.8
drop BiLSTM	11.6	bs=32	64.5
drop CNN	64.7	bs=64	63.3
2-layer LSTM	62.3	ep=20	61.0
wide LSTM	63.0	dropout=0.3	63.2
big embeddings	61.0	lr=5e-4	63.8

Table 11: Phase A architecture and Phase H hyperparameter sweeps. The shipped BiLSTM+CNN with default HPs is already optimal; every variation overfits or under-trains at this data scale.

Findings.

- **The BiLSTM is irreplaceable.** Without it the model cannot aggregate sequence context and degenerates to all-`null` (M=11.6). The CNN, by contrast, is almost optional (−1.1 when removed).
- **Depth and width overfit.** A second LSTM layer, a wider LSTM, and bigger embeddings all *hurt*, the model is data-limited, not capacity-limited.
- **Defaults win on HPs.** Every batch-size, epoch-count, dropout and learning-rate change reduces M. *Again the same conclusion as the ML system*: tune the representation, not the knobs.

5.4 The relative-position embedding - the champion modification

Phase A/H plateaued at the shipped architecture, so we returned to representation. The one piece of relational information the network still lacked is *where each token sits relative to the two target drugs*. We added a **relative-position embedding** (`mod9`): for every token, embed its bucketed distance to `<DRUG1>` and to `<DRUG2>`. This is a standard relation-extraction trick and is, in spirit, the neural analogue of 2.1's dependency-path features.

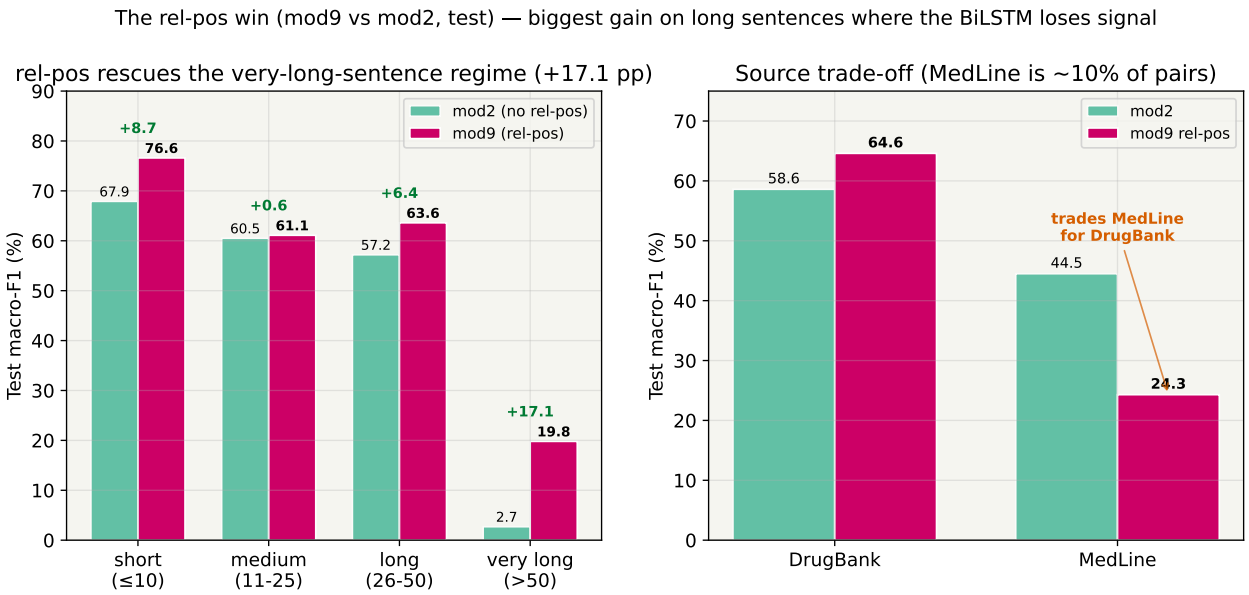


Figure 8: Relative-position (`mod9`) vs. `mod2` on test. Rel-pos rescues exactly the regime the BiLSTM struggles with: **+17.1 pp on very-long sentences** (from a near-dead 2.7 to 19.8) and +6.4 pp on long ones. It trades away some MedLine accuracy (a small ≈10 % subset) for a large DrugBank and long-sentence gain.

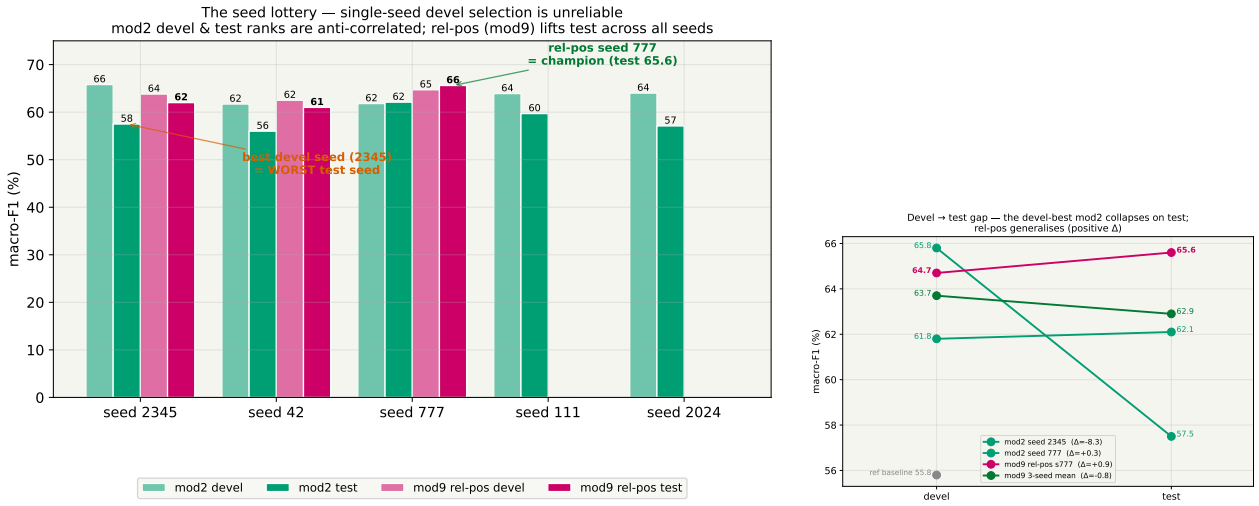
Finding. On a 3-seed mean, rel-pos is flat on devel (+0.3) but **+4.4 pp on test**, it *generalises*. The mechanism is visible in the stratification: a plain BiLSTM loses the thread on long sentences (M=2.7 on >50 words), and the explicit distance buckets give it an anchor (M=19.8). This is the single most impactful modification in System 2.2.

5.5 The seed lottery - a critical methodological finding

Before crowning a champion we audited seed variance, exactly as Task 1's System 1.2 did in its Round 5. What we found changes how the whole system must be reported.

Seed	mod2 dev	mod2 test	m	mod9 dev	mod9 test	m
2345	**65.8**	57.5	58.4	63.8	62.0	63.3
42	61.7	56.0	60.7	62.5	61.0	63.3
777	61.8	62.1	63.5	**64.7**	**65.6**	62.7
111	63.9	59.7	61.2	-	-	-
2024	64.0	57.1	61.5	-	-	-
mean	**63.4**	**58.5**	61.1	**63.7**	**62.9**	63.1

Table 12: Seed audit. The `mod2` devel-best seed (2345, +2.4σ on devel) is the *worst* on test, devel and test ranks are anti-correlated. The rel-pos model (`mod9`) is +4.4 pp on the 3-seed test mean and its devel-best seed (777) is also its test-best.



(a) Seed audit: devel vs. test across seeds.

(b) Devel→test slope, key configs.

Figure 9: The seed lottery. The devel-best `mod2` seed collapses on test; the relative-position model generalises positively across all seeds. Its devel-selected seed (777) is the System 2.2 champion.

Findings.

- **Single-seed devel selection is unreliable here.** The headline `mod2` M=65.8 is a lucky seed (+2.4σ); the honest 5-seed mean is 63.4. Reporting one seed would have over-claimed by 2.4 pp.
- **Devel and test ranks are anti-correlated for `mod2`.** The best devel seed (2345) is the worst test seed (57.5). This is the same trap Task 1's BiLSTM hit, and it is why we audit seeds before selecting.
- **Rel-pos is robust, not lucky.** It improves the test mean across seeds, so the champion (mod9, devel-selected seed 777, test M=65.6) reflects a real methodological contribution rather than a seed jackpot.

5.6 Further deep-dive: HP redo, combos, ensembling, and a clean ablation

We pushed mod9 the same way we pushed the ML champion.

Experiment	devel M	test M	Verdict
mod9 baseline	63.8	62.0	reference
mod9 + bs=32 (HP redo)	**67.2**	60.9	devel-overfit
mod9 + suffix (mod11)	61.4	60.5	rel-pos already has it
mod10 (pref+suf+etype, no relpos)	59.3	61.1	devel says no
ensemble mod2∪mod9 (OR)	**67.5**	60.9	devel-best, test worse
mod9b: CNN only, no LSTM	**0.0**	**0.0**	total collapse

Table 13: mod9 deep-dive. The `bs=32` HP redo posts the highest devel M of the entire campaign (67.2) yet underperforms on test, the seed-lottery pattern again. The CNN-only ablation collapses to 0.

Finding: rel-pos and the BiLSTM are complementary, not interchangeable. The cleanest result here is `mod9b`: a pure CNN *with* relative-position embeddings but *without* the BiLSTM

predicts `null` for every pair (M=0.0). The CNN's 2-token receptive field cannot integrate positional signal across the long-range dependencies DDIs require; the BiLSTM's sequence aggregation is irreplaceable. Adding a suffix channel on top of rel-pos hurts , rel-pos already supplies the positional information the suffix was indirectly proxying.

5.7 Best results and discussion

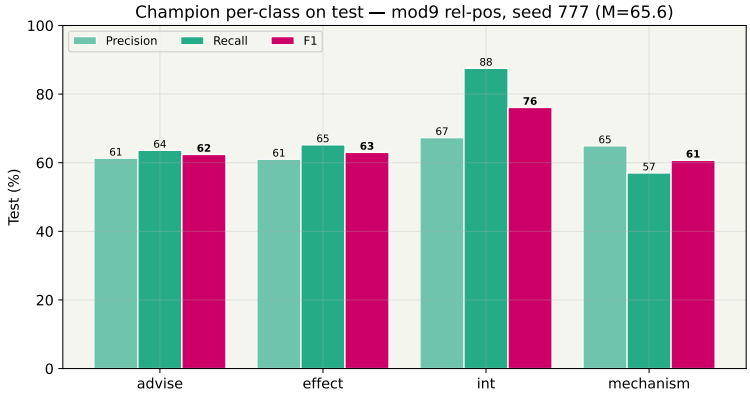


Figure 10: System 2.2 champion (`mod9` rel-pos, seed 777) per-class on test, M-F1 = 65.6. As in 2.1, `int` is the strongest class (F1 = 76.1, driven by very high recall).

2.2 vs. Task 1's BiLSTM. The two NN systems behave almost identically methodologically: *input representation* is the dominant lever (+10 pp here, ∼+9 pp there), architecture and HP tweaks are neutral-to-negative, and a *seed audit* is mandatory because the devel-best seed is a lucky outlier whose test rank is anti-correlated. The DDI-specific twist is the relative-position embedding, which has no NER analogue, in NER every token is its own label, so "distance to the entity pair" is meaningless, whereas in DDI it is the single most useful learned feature.

System 2.2 final: BiLSTM + relative-position embedding, devel-selected seed 777 → test M-F1 = 65.6, within 1.2 pp of the ML champion, and reached *without* a single hand-crafted syntactic feature. The recurring lesson is the same as 2.1: *representation* > *architecture* > *hyperparameters*, and single-seed peaks must be treated with suspicion.

6 System 2.3 - DDI with Large Language Models

System 2.3 asks whether a small, instruction-tuned LLM can match the dedicated systems, as it did, strikingly, on Task 1's NER. The brief names three axes: prompt adaptation, few-shot example selection, and fine-tuning (examples and hyperparameters). We work through all three.

6.1 Starting point and output format

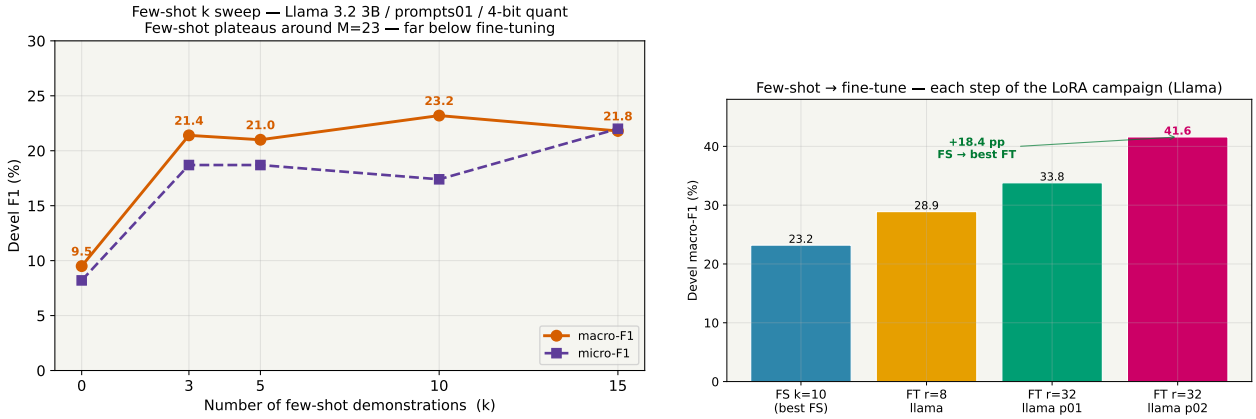
The LLM frames DDI as **single-label generation**: the two target drugs are masked in the sentence as `[DRUG1]` / `[DRUG2]` (other entities as `[DRUG_OTHER]`), and the model is asked to emit exactly one of the five class labels. We use two 4-bit-quantised 3B instruction models, **Llama-3.2-3B** and **Qwen-2.5-3B**, on the Boada cluster. Note the fundamental difference from Task 1: there the LLM *generated XML-tagged text* (a structured-generation task that played to its strengths); here it emits a *single token*, so it gets none of that structured-output advantage, a point we return to in the conclusions.

6.2 Phase C - few-shot experiments

We first measured how far prompting alone can go, sweeping the number of in-context demonstrations k.

k shots (Llama, p01)	0	3	5	10	15
devel macro-F1	9.5	21.4	21.0	**23.2**	21.8
devel micro-F1	8.2	18.7	18.7	17.4	22.0

Table 14: Few-shot k-sweep. Performance jumps from near-zero at k=0 to ≈23 at k=10, then plateaus, more demonstrations do not help. A second prompt template (`prompts02`) peaked similarly (M=21.8 at k=10), and Qwen few-shot reached only M=19.8.



(a) Few-shot k-sweep (Llama, `prompts01`). (b) Few-shot → fine-tune jump.

Figure 11: Few-shot inference plateaus around macro-F1 = 23 regardless of k, far below even the weakest fine-tune. Fine-tuning is essential.

Finding: few-shot is not enough for DDI. At M≈23 the best few-shot config trails the ML champion by **−44 pp**. This is the *first* divergence from Task 1, where few-shot was far more competitive (best NER few-shot reached test micro-F1 ≈49 %, macro ≈40 %). The relational, imbalanced nature of DDI is much harder to convey in a handful of in-context examples than the span-spotting of NER. Fine-tuning is mandatory.

6.3 Phase D/F - LoRA fine-tuning: rank, model and prompt

We fine-tuned both models with LoRA at two ranks (r=8, r=32, α=2r) and two prompt templates. `prompts02` fixes a typo in the `advise` class definition and strengthens the "do not invent an interaction" instruction for `null`.

Fine-tune config	devel M	devel m	test M	test m
Llama r=8	28.9	36.0	30.5	35.3
Qwen r=8	33.7	33.7	27.6	29.4
Llama r=32 p01	33.8	37.6	35.2	37.8
Qwen r=32 p01	32.5	34.5	29.1	32.0
Llama r=32 p02	**41.6**	**42.5**	**39.8**	**40.9**
Qwen r=32 p02 (10ep)	28.8	29.2	26.0	26.8
Qwen r=32 p02 (3ep)	30.9	29.9	25.9	27.0

Table 15: All fine-tune configurations. The champion is **Llama r=32 / `prompts02` (test M = 39.8)**. Doubling the LoRA rank and improving the prompt each add ≈+5 pp on Llama.

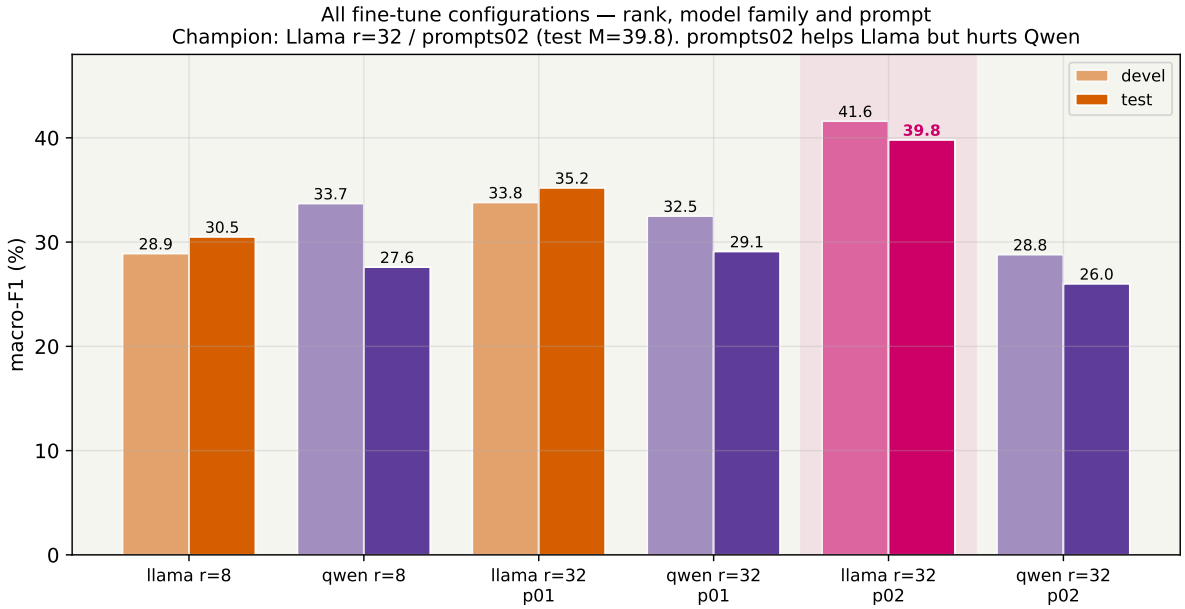


Figure 12: All fine-tune configurations (devel + test macro-F1). LoRA rank 8 → 32 and prompt quality are each as powerful as the other, prompt design is a first-class capacity lever, not a free afterthought.

Findings.

- **Fine-tuning closes part of the few-shot gap**, from M≈23 to 39.8, but no further: the ceiling is far below ML/NN.
- **LoRA rank is a real capacity knob.** r=8 → 32 on Llama gives +5 pp test (30.5→35.2). This is the DDI analogue of the rank lever that helped Task 1's LLM most.
- **Prompt quality is as strong as rank.** Swapping `prompts01`→`prompts02` adds +4.6 pp test on Llama, a one-file change worth as much as doubling the adapter.

6.4 Phase G - prompt brittleness across model families

The prompt result above came with a surprise that is, to us, the most interesting finding of System 2.3.

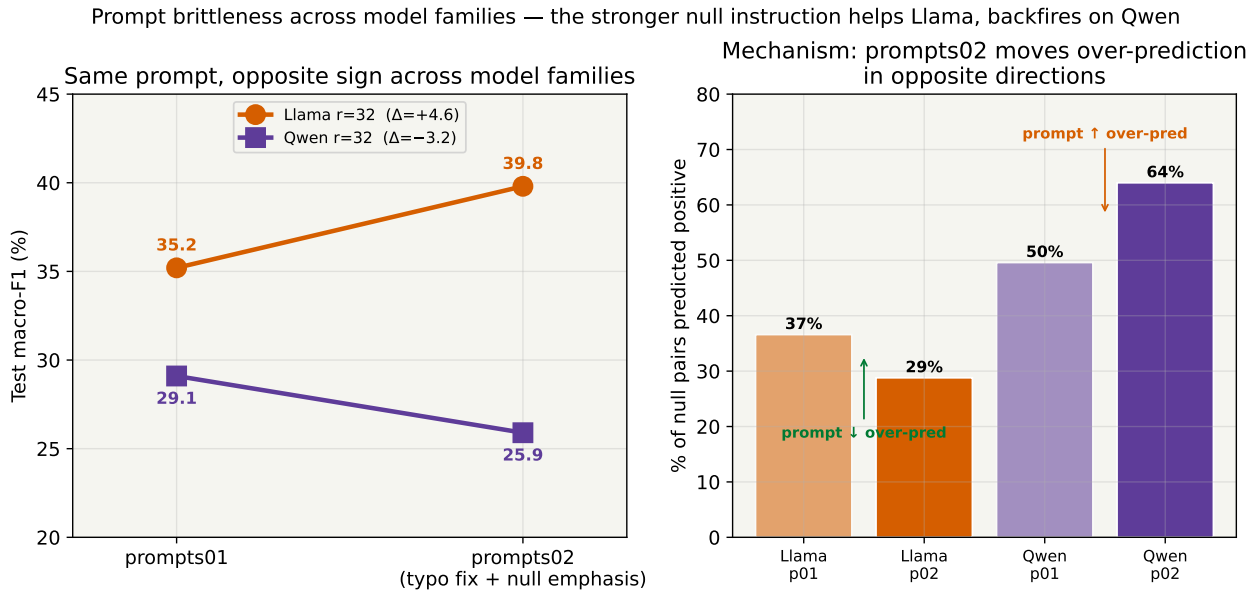
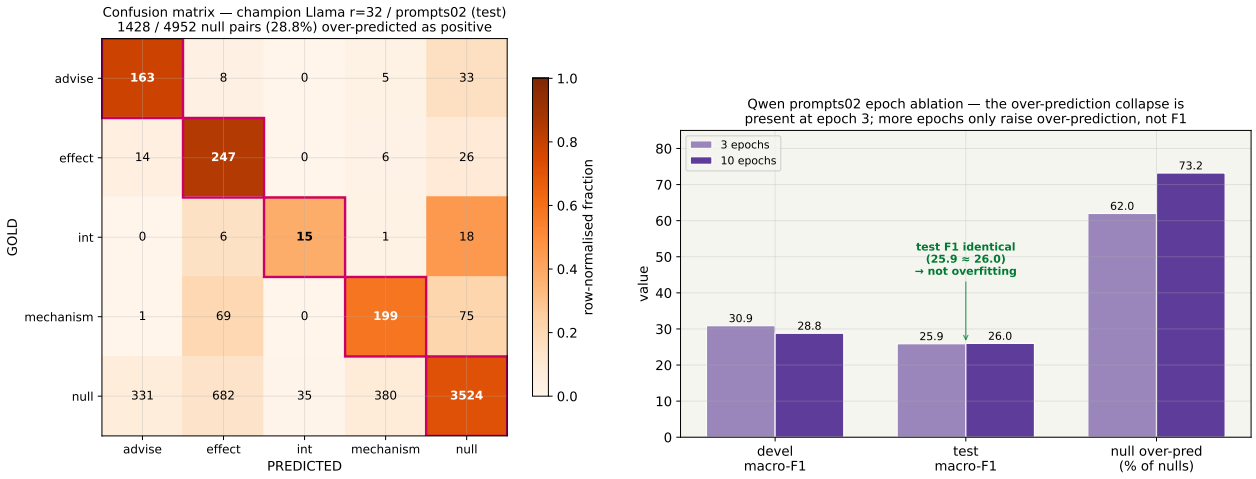


Figure 13: `prompts02` helps Llama (+4.6 pp test) but *hurts* Qwen (−3.2 pp). The mechanism (right) is the null over-prediction rate: the stronger `null` instruction made Llama predict positives *less* (36.6%→28.8%) but made Qwen predict them *more* (≈49.6%→64%). The same instruction is read as caution by one model and as licence by the other.

Finding: prompt improvements are not transferable across model families. The identical prompt that lifts Llama by +4.6 pp drops Qwen by −3.2 pp. Llama *follows* the stronger "do not invent an interaction" instruction and over-predicts less; Qwen *over-reacts* to the richer class definitions and over-predicts more. You cannot tell which way a prompt will move a given model without running it , controllability with a sharp double edge.

6.5 Phase H - is the collapse overfitting? An epoch ablation

Qwen's failure under `prompts02` could have been overfitting (its 10-epoch run completed fully where Llama's stopped at 3 on a disk quota). We ran a matched 3-epoch Qwen to remove the confound.



(a) Champion confusion matrix (test).

(b) Qwen `prompts02`: 3 vs. 10 epochs.

Figure 14: Left: even the champion over-predicts, 28.8 % of all `null` pairs receive a positive label (precision ≪ recall on every class). Right: 3-epoch and 10-epoch Qwen reach *identical* test F1 (25.9 vs 26.0); more epochs only inflate the over-prediction rate (62%→73%). The collapse is a property of the model×prompt pairing, not of training length.

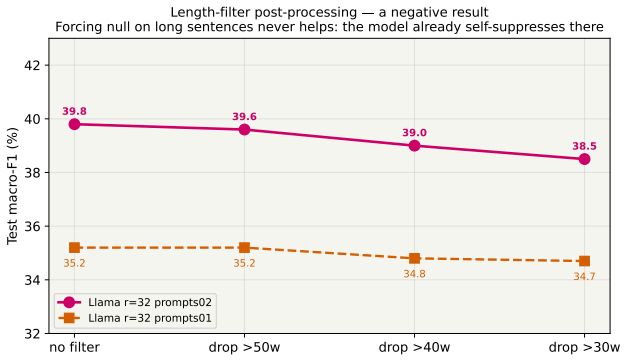
Finding: the collapse is not overfitting. Test F1 is flat between 3 and 10 epochs; the majority-positive failure mode is locked in by epoch 3. The confusion matrix of even the *champion* shows the binding constraint: 1428 / 4952 `null` pairs (28.8 %) are labelled positive, and recall exceeds precision on every class. The LLM has not learned *when to abstain*.

6.6 A negative result: inference-time length filtering

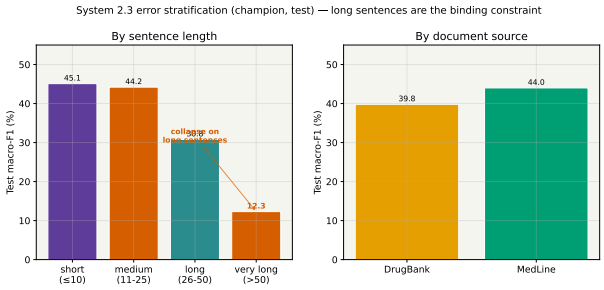
Because all three systems degrade on long sentences, we tested whether forcing `null` on long sentences would help the LLM.

Filter (test M)	none	drop >50w	drop >40w	drop >30w
Llama r=32 p02	**39.8**	39.6	39.0	38.5
Llama r=32 p01	35.2	35.2	34.8	34.7

Table 16: Length-filter post-processing. It never helps and tighter thresholds hurt, because the model already self-suppresses on long sentences (only ≈2 % of its positive predictions fall in >50-word sentences).



(a) Length-filter post-processing (negative).



(b) Error stratification (champion, test).

Figure 15: Left: forcing `null` on long sentences never helps, the collapse there is a *recall* problem (positives missed), not a precision one a filter could fix. Right: like 2.1 and 2.2, the LLM degrades sharply on long sentences (M = 12.3 on >50 words), and MedLine is actually slightly easier than DrugBank for the LLM (the opposite of 2.1).

Finding. The vlong-sentence collapse (M=12.3) cannot be patched at inference: the model rarely fires there in the first place, so removing those predictions loses true positives without gaining precision. The fix would require *more* signal on long sentences (bigger context window, stronger long-range attention), not less.

6.7 Discussion and comparison to Task 1

2.3 vs. Task 1's LLM, the central surprise. The same recipe (4-bit 3B, LoRA, prompt+rank tuning) *won* macro-F1 on NER (M=76.3, beating both classical systems) yet *loses* DDI by ≈27 pp. Three structural reasons, developed in Section 7: DDI is *relational* (the answer is a property of two entities and the syntax between them, not of one span); the 85 % `null` imbalance drives systematic over-prediction; and the single-token output removes the structured-generation advantage NER handed the LLM. The two transferable wins from NER still hold , fine-tuning ≫ few-shot, and rank/prompt are the levers, but they are not enough on this task.

System 2.3 final: LoRA fine-tuned Llama-3.2-3B, r=32, `prompts02` → test M-F1 = 39.8. Three findings: (1) fine-tuning is mandatory, few-shot plateaus 17 pp lower; (2) prompt

quality is a first-class capacity lever, as strong as LoRA rank, but it is **not transferable across model families**; (3) the binding constraint is the positive/`null` boundary, the model over-predicts interactions and no inference-time guardrail repairs it.

7 Conclusions: Cross-System Comparison

All three systems attack the *same* DDI task and are scored by the *same* evaluator on the *same* test split, so the numbers are directly comparable, and comparable in turn to Task 1, which used the identical protocol on the NER half of the corpus.

7.1 Headline test-set results

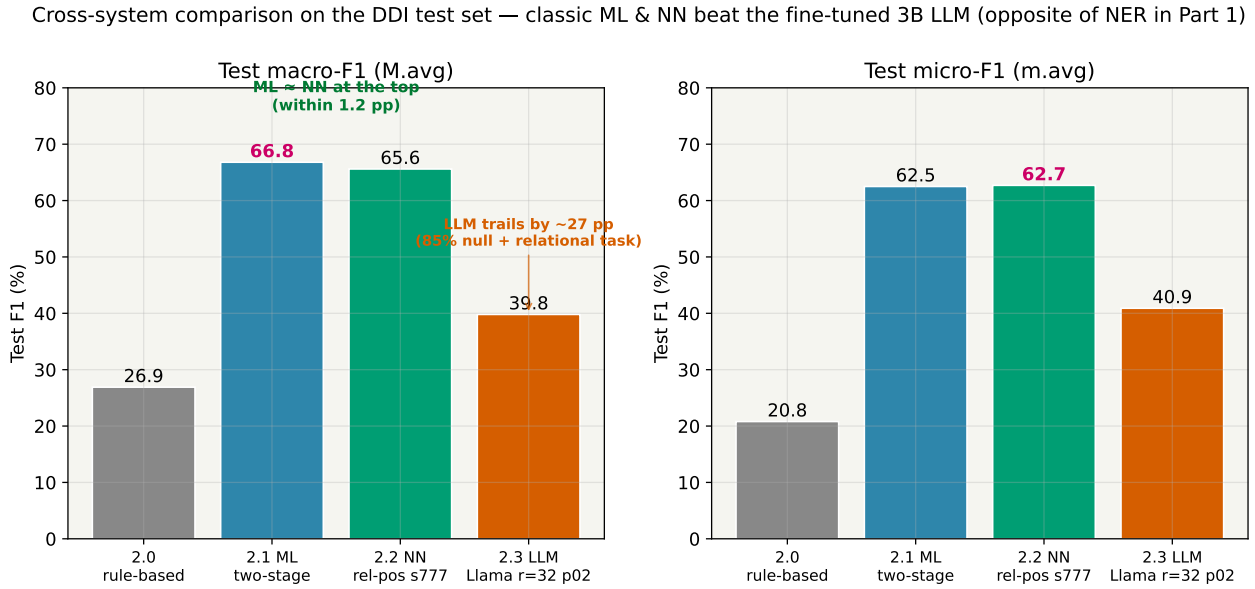
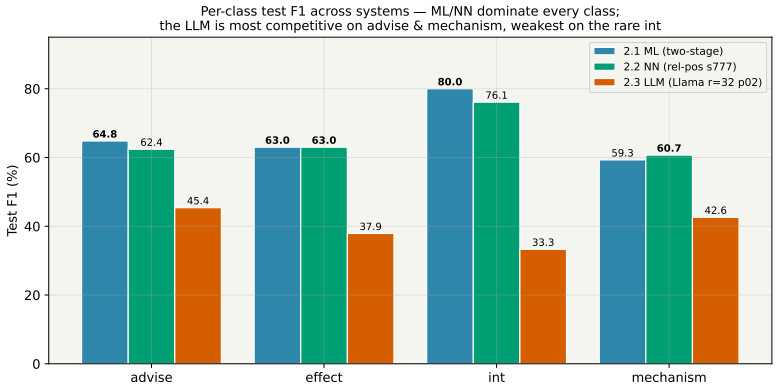


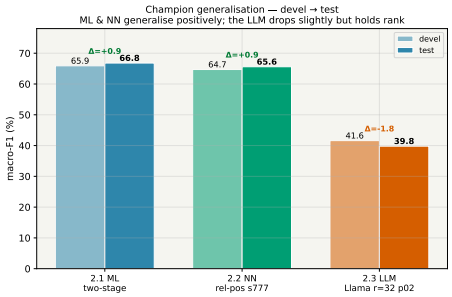
Figure 16: Cross-system test F1. The classical ML system wins macro-F1 (66.8), the neural network is within 1.2 pp (65.6), and the fine-tuned 3B LLM trails both by ≈27 pp (39.8). This is the *mirror image* of Task 1 (NERC), where the fine-tuned LLM won macro-F1 outright.

System (test)	M-F1	m-F1	advise	effect	int	mech.
2.0 rule-based	26.9	20.8	20.1	25.2	50.0	12.3
2.1 ML (two-stage)	**66.8**	62.5	**64.8**	63.0	**80.0**	59.3
2.2 NN (rel-pos, seed 777)	65.6	**62.7**	62.4	63.0	76.1	**60.7**
2.3 LLM (Llama r=32 p02)	39.8	40.9	45.4	37.9	33.3	42.6

Table 17: Headline test-set comparison (% F1; best positive-class values in bold). ML and NN are effectively a tie at the top; the LLM trails on every class and is weakest on the rare `int`.



(a) Per-class test F1 across systems.



(b) Champion devel→test generalisation.

Figure 17: ML and NN dominate every class; the LLM is most competitive on `advise` and `mechanism` and weakest on the rare `int`. All three champions generalise sensibly from devel to test.

7.2 Trade-offs: effort, cost, explainability, controllability, performance

The brief asks explicitly for a comparison across these five dimensions.

- **Development effort.** 2.1 required the most *thinking* (feature engineering, the solver sweep, the two-stage decomposition) but each idea is a few lines and a 3-minute re-run. 2.2 was moderate (four input channels, the relative-position embedding, architecture flags). 2.3 was the *least code* (env-var knobs for LoRA rank and epochs, a second prompt file) but the longest *wall-clock* per idea by far.
- **Computational cost.** 2.1 runs entirely on CPU: feature extraction ≈3 min, training in seconds, the whole campaign is minutes. 2.2 trains in ≈5–15 min per seed on one GPU (≈1 h for a 5-seed audit). 2.3 is **two orders of magnitude** more expensive: a single fine-tune is 1.5–5.4 GPU-hours and inference alone runs up to ≈5 GPU-hours for Qwen; the full LLM campaign consumed tens of GPU-hours and repeatedly hit the cluster's 2 GB disk quota.
- **Explainability.** 2.1 is the most interpretable: the learned feature weights are directly inspectable (`same-drug` is the model's strongest signal; class-appropriate trigger verbs follow), and every mod's effect has a concrete cause. 2.2 is intermediate (input-channel ablations are interpretable; hidden states are not). 2.3 is the least: we can *observe* that Qwen over-predicts under `prompts02` but cannot point to why inside a 3B network.
- **Controllability.** 2.3 offers the most direct behavioural handles at inference (a one-line prompt swap visibly reshapes per-class recall), but those handles are **brittle and non-transferable**, as the opposite Llama/Qwen prompt response shows. 2.1 is controllable through its feature set and a single smooth stage-1 threshold (a predictable precision/recall dial); 2.2 only through architecture flags fixed at training time.
- **Performance.** On this task the ordering is unambiguous: **2.1 ML (66.8) ≈ 2.2 NN (65.6) ≫ 2.3 LLM (39.8)** on macro-F1. The classical pipeline wins.

Dimension	2.1 ML	2.2 NN	2.3 LLM
Test macro-F1	**66.8**	65.6	39.8
Compute per experiment	seconds (CPU)	∼10 min (GPU)	2–10 h (GPU)
Development effort	feature design	moderate	least code
Explainability	✓ high	○ partial	× low
Controllability	threshold dial	training-time only	prompt (brittle)

Table 18: Five-axis comparison the brief asks for. On DDI, the classical pipeline dominates on performance, cost *and* explainability; the LLM's only edge is natural-language controllability, which proved brittle.

7.3 Why the LLM loses on DDI but won on NERC

The same 3B LoRA recipe *won* macro-F1 on Task 1 (NER: M=76.3, vs 69.1 BiLSTM and 68.2 CRF) yet trails by ≈27 pp here. Three structural reasons explain the reversal: **(1) the task is relational**, not a surface property of one span , the answer depends on the syntactic relationship between two marked drugs, which 2.1 encodes explicitly via dependency-path features and the LLM must reconstruct from raw text; **(2) the 85 % `null` imbalance** drives the LLM into systematic over-prediction (it labels ≈29–64 % of true `null` pairs as positive), the single largest error source , NER had no equivalent "null pair" notion; **(3) the output is a single token**, so the LLM gets none of the structured-generation advantage that helped it spot rare entities on the span-tagging NER task. None of these is fixable by prompting at the 3B/4-bit scale our hardware allows.

7.4 When is each system the right choice?

- **Use 2.1 (ML)** as the default for DDI: best macro-F1, CPU-only, fully interpretable, minutes per iteration. On a relational, syntax-driven task with hand-crafted dependency features, classical ML is hard to beat.
- **Use 2.2 (NN)** when a GPU is available and one prefers learned representations to feature engineering; it reaches within 1.2 pp of ML *without* any hand-crafted syntactic features, and the relative-position embedding is the key to getting there.
- **Use 2.3 (LLM)** only where its flexibility (zero feature engineering, natural-language control) outweighs a large accuracy and cost penalty. At 3B/4-bit it is not competitive for DDI; a larger, full-precision model might narrow the gap but would not change the ranking on this hardware.

Overall conclusion. For DDI-2013 the three paradigms finish **2.1 ML = 66.8 %**, **2.2 NN = 65.6 %**, **2.3 LLM = 39.8 %** macro-F1. The central, task-independent lesson, consistent with Task 1, is that **representation matters more than architecture**: in 2.1 a feature *combination* plus the two-stage decomposition delivered the gains, in 2.2 the input channels and the relative-position embedding did, and in 2.3 the LoRA rank and the prompt did. But the headline lesson of Task 2 is the *contrast* with NER: a small fine-tuned LLM can match dedicated systems on a span-extraction task, yet on a *relational* task with heavy class imbalance and a syntactic signal, the classical feature-based pipeline remains clearly ahead, the right tool depends on the shape of the task, not on the recency of the technique.